

สรุปการทำงานของคำสั่งใน main.py

เอกสารนี้อธิบายโปรแกรม To-Do List ที่ทำงานผ่านเมนูในเทอร์มินัล เขียนด้วยภาษา Python และบันทึกรายการงานไว้ในไฟล์ tasks.json

1. ภาพรวมการทำงาน

เมื่อเริ่มโปรแกรม ฟังก์ชัน main() จะอ่านรายการงานจาก tasks.json เริ่มเธรดตรวจสอบการแจ้งเตือน และแสดงเมนูจนกว่าผู้ใช้จะเลือก Exit ผู้ใช้สามารถเพิ่ม แสดง ทำเสร็จ หรือลบงานได้ โดยการเปลี่ยนแปลงจะถูกบันทึกกลับลงไฟล์

ลำดับการทำงานหลัก

- เริ่มโปรแกรม
 - > load_task() อ่าน tasks.json
 - > เริ่ม reminder_loop() ในเธรดเบื้องหลัง
 - > แสดงเมนูและรับ choice
 - > ทำงานตามตัวเลือก 1-5
 - > วนกลับไปแสดงเมนู หรือจบเมื่อเลือก 5

2. ไบรารีและตัวแปรสำคัญ

ส่วนประกอบ	หน้าที่
json	อ่านและเขียนข้อมูลรายการงานในรูปแบบ JSON
time	หยุดการทำงานของเธรด 1 วินาทีระหว่างตรวจสอบเตือน
datetime	ตรวจสอบรูปแบบวันเวลาและเปรียบเทียบเวลาปัจจุบัน
threading	สร้างเธรดตรวจสอบแจ้งเตือนแยกจากเมนูหลัก
ZoneInfo	กำหนดเขตเวลา Asia/Bangkok
TASK_FILE	เก็บชื่อไฟล์ข้อมูล คือ tasks.json
tasks	ลิสต์ที่เก็บรายการงานทั้งหมดในหน่วยความจำ
tasks_lock	ล็อกเพื่อป้องกันการอ่านหรือเขียน tasks พร้อมกันจากหลายเธรด

3. อธิบายฟังก์ชัน

reminder_loop() — วนทำงานตลอดเวลา เรียก check_reminders() แล้วหยุด 1 วินาทีด้วย time.sleep(1) ฟังก์ชันนี้ทำงานในเธรด daemon จึงไม่ขวางการปิดโปรแกรมเมื่อ main thread จบ

check_reminders() — อ่านเวลาปัจจุบันตามเขตเวลา Bangkok แล้วตรวจงานทีละรายการ โดยข้ามงานที่ reminded หรือ complete เป็น True จากนั้นแปลง date และ time เป็น datetime หากถึงกำหนดจะแสดงข้อความเตือน ตั้ง reminded เป็น True และบันทึกข้อมูล

load_task() — เปิด tasks.json แล้วใช้ json.load() แปลงข้อมูล JSON เป็นลิสต์ Python จากนั้นส่งค่ากลับให้ main()

`save_task()` — ล็อก `tasks_lock` เปิด `tasks.json` ในโหมดเขียน และใช้ `json.dump()` บันทึกลิสต์ `tasks` ด้วย `indent=4`ให้อ่านง่าย

`get_task_number(prompt)` — รับหมายเลขงานจากผู้ใช้ แปลงเป็นจำนวนเต็ม ตรวจสอบว่าหมายเลขอยู่ในช่วง 1 ถึงจำนวนงาน แล้วคืนค่าเป็นดัชนีแบบเริ่มที่ 0 หากข้อมูลไม่ถูกต้องจะพิมพ์ข้อความแจ้งเตือนและคืน `None`

`main()` — เป็นศูนย์กลางของโปรแกรม โหลดข้อมูล เริ่มเรดแจ้งเตือน แสดงเมนู และจัดการคำสั่งของผู้ใช้จนกว่าจะเลือก 5

4. รายละเอียดเมนูใน main()

ตัวเลือก	การทำงาน
1. Add Tasks	รับชื่อ รายละเอียด วันที่ และเวลา ตรวจสอบรูปแบบด้วย %Y-%m-%d %H:%M แล้วเพิ่มดิกชันนารีงานใหม่ลงใน tasks โดยเริ่ม complete และ reminded เป็น False
2. Show Tasks	วนแสดงทุกงานพร้อมหมายเลข สถานะ เครื่องหมาย เมื่อทำเสร็จ ชื่องาน วันที่ และเวลา
3. Complete Tasks	เรียก get_task_number() รับหมายเลขงาน ตั้งค่า complete เป็น True แล้วบันทึกไฟล์
4. Delete Tasks	เรียก get_task_number() รับหมายเลขงาน ลบด้วย tasks.pop(index) แล้วบันทึกไฟล์
5. Exit	ออกจาก while loop ทำให้ main() จบการทำงาน ส่วน reminder thread เป็น daemon จึงหยุดตามโปรแกรม

5. โครงสร้างข้อมูลใน tasks.json

แต่ละงานเป็นออบเจ็กต์ที่มีคีย์ title, des, date, time, complete และ reminded
ตัวอย่างโครงสร้างที่โปรแกรมสร้างเมื่อเพิ่มงานคือ:

```
{
  "title": "อ่านหนังสือ",
  "des": "บทที่ 1",
  "date": "2026-09-23",
  "time": "18:30",
  "complete": false,
  "reminded": false
}
```

6. ระบบแจ้งเตือนและการทำงานหลายเธรด

main() สร้างเธรดด้วย threading.Thread(target=reminder_loop, daemon=True) แล้วเริ่มด้วย start()
ทำให้ระบบแจ้งเตือนตรวจงานได้ทุก 1 วินาทีขณะที่ผู้ใช้อยู่ยังเลือกเมนูได้ตามปกติ ก่อนเข้าถึง tasks ใน
check_reminders() และ save_task() โปรแกรมใช้ with tasks_lock เพื่อป้องกันข้อมูลชนกันระหว่างเธรด

7. การตรวจสอบข้อผิดพลาด

การเพิ่มงานตรวจวันเวลาและปฏิกิริยาที่ตรงรูปแบบ
หากกรอกหมายเลขงานไม่ใช่ตัวเลขหรืออยู่นอกช่วงจะไม่แก้ไขข้อมูล ส่วน check_reminders() จัดการ KeyError, TypeError และ ValueError โดยพิมพ์ข้อความ Invalid date/time แล้วข้ามงานนั้น

8. ข้อสังเกตจากโค้ดปัจจุบัน

- ฟังก์ชัน save_task() ใช้ไฟล์ชื่อ tasks.json แบบ relative path ดังนั้นควรรันโปรแกรมจากโฟลเดอร์โปรเจกต์
- โปรแกรมไม่ได้ตรวจโครงสร้าง JSON ตอน load_task() หากไฟล์เสียหายอาจเกิดข้อผิดพลาดก่อนแสดงเมนู
- ช่อง des ถูกบันทึกและรับค่า แต่เมนู Show Tasks ยังไม่ได้แสดงรายละเอียดนี้
- เมื่อแจ้งเตือนแล้วจะตั้ง reminded เป็น True เพื่อไม่ให้เตือนซ้ำ แม้ผู้ใช้อยู่ยังไม่ได้ทำงานเสร็จ

สรุป: main.py เป็นโปรแกรมจัดการงานแบบโต้ตอบที่ใช้ JSON เป็นที่เก็บข้อมูล และใช้เธรดเบื้องหลังสำหรับแจ้งเตือนตามกำหนดเวลา